



Linked List Tutorial Solutions

1. Check for free node:

- Check if the nextfree pointer (head of free node list) is null.
- If null, display an error message: "No free node available" and terminate the program.

Allocate new node:

- Take the node pointed by nextfree from the free list.
- Update nextfree to point to the next free node in the list (nextfree = nextfree→next).
- Copy the new element's data into this allocated node.
- Set this node's next pointer to null.
- *(Draw diagram: Free list before and after allocation, showing nextfree moving forward.)*

Update pointers:

- If the linked list is empty (head == null), set head to point to the new node.
- Otherwise, traverse the linked list from head by following the next pointers until reaching the last node (where current→next == null).

Insert the new node:

- Set the next pointer of the last node to point to the newly allocated node.
- *(Draw diagram: Linked list before and after insertion, showing new node appended.)*



2. (a) Show the following in diagram:
- The **head/start** of the linked list
 - The **pointers between nodes**
 - Nodes arranged in **alphabetical order**
 - A **null pointer** at the end of the linked list
 - A separate **free space list**

(b) (i) Add "Jenny" into the linked list

Allocate Node:

- Copy the data "Jenny" into the node pointed to by the free space pointer (NextFree).
- Assign this node to a pointer variable (newNode = NextFree).
- Update NextFree to point to the next available free node in the free list (NextFree = NextFree → next).

Traverse the List:

- Starting from the Head, traverse the linked list by comparing each node's data alphabetically with "Jenny".
- Stop when you find a node whose next node contains "Nelson", as "Jenny" should be inserted before "Nelson".

Update Pointers:

- Change the pointer of the node currently pointing to "Nelson" so that it now points to the "Jenny" node (previous → next = newNode).
- Set "Jenny" 's next pointer to point to "Nelson" (newNode → next = current).

(b) (ii) Remove "Henry" from the linked list

Traversing the List to Find "Henry":

- Start from the Head of the linked list.
- Use a temporary pointer (e.g., current) to move through the list.
- Use another pointer (previous) to keep track of the node just before current.
- Continue traversing until current → data == "Henry".
 - At this point, current points to the node containing "Henry", and previous points to the node before "Henry".

Changing Pointers to Remove "Henry":

- To delete "Henry", bypass it in the linked list:
 - Set previous → next = current → next
 - This removes "Henry" from the list by skipping over it.

Return Deleted Node to Free Space List:

- Set the deleted node's next pointer to point to the current start of the free space list (current → next = NextFree).
- Update NextFree to point to the deleted node (NextFree = current).
- This places the deleted node at the front of the free space list.



3. (a) **Write pseudocode and explain**

<pre>FUNCTION count_nodes: count ← 0 current ← start WHILE current is not NULL count ← count + 1 current ← current.pointer ENDWHILE RETURN count ENDFUNCTION</pre>	Start at the head node of the linked list. Initialise a count variable to 0 and a current pointer to start pointer. Use the current pointer to traverse the linked list, until the end where current pointer points to null. For each node encountered during traversal, increment count variable by 1. Return the count as the total number of nodes in the linked list.
--	--

(b)

<pre>PROCEDURE update_data(target, new_value) current ← start WHILE current is not NULL IF current.data ← target current.data ← new_value BREAK ENDIF current ← current.pointer ENDWHILE ENDPROCEDURE</pre>	Traverse linked list to find the target node which contains the given value. Update the data item of the target node with the new value
---	--

(c) Take the last node of the first linked list and set its pointer to point to the start of the second linked list.

```
PROCEDURE append_linked_list(another):
    IF start is NULL // first linked list is empty,
                    // just point its start pointer
                    // to the first node of the second linked list
        start ← another.start
    ELSE
        current ← start
        WHILE current.pointer is not NULL
            current ← current.pointer
        ENDWHILE
        current.pointer ← another.start
    ENDIF
ENDPROCEDURE
```



(d) Assume split in the middle of linked list. Run an algorithm to count the number of nodes in the linked list. Then the middle node can be found by dividing by total number of nodes by two.

Traverse the original linked list to the middle node.

Create a new linked list starting from the middle node, and set it as the start of the second linked list.

Update the pointer of the last node before the middle node to be NULL to break the original linked list into two separate lists.

```
PROCEDURE split_linked_list
    count = count_nodes() //use function to count number of nodes
    IF count < 2          // Cannot split if linked list has 0 or 1 node
        RETURN
    ENDIF
    middle_index ← TO_INTEGER(count DIVIDE 2)
    previous ← NULL
    current ← start
    FOR index ← 0 TO middle_index
        previous ← current
        current ← current.pointer
    ENDFOR

    prev.pointer ← NULL // Update pointer of last node before
                        // middle to NULL
    create new_linked_list //Create new linked list
    new_linked_list.start ← current //starting from the middle node
    RETURN new_linked_list
ENDPROCEDURE
```